ANTIGRAVITY BUILD DOCUMENT v2.0 | CyberHack V4 2025

CYBERHAWK

Offline AI-Powered Digital Forensics Platform

Built on Servos + Orca + Orchestrata | Powered by Local LLMs

@security-auditing @rag-engineer @react-patterns @python-patterns @docker-expert

@typescript-expe @vulnerability-sc @api-security-best-practice @test-driven-development

---

UI COMPONENTS FROM 21st.dev

halide-topo-hero • gooey-text-morphing • ultra-quality-showcase-grid • spatial-product-showcase

---

Document Version:	2.0.0 – Antigravity Edition
Build Target:	Antigravity IDE / Claude Code / Cursor
Skills Registry:	antigravity-awesome-skills v4.11.0 (713 skills)
Prompt Source:	x1xhlol/system-prompts-and-models-of-ai-tools
Team:	Team MoMoSapiens | Akash Santhnu Sundar
Hackathon:	CyberHack V4 2025 | Offline LLM Investigation Track
Pages:	45+

HOW TO USE THIS DOCUMENT WITH ANTIGRAVITY

```
Step 1. Install skills:  npx antigravity-awesome-skills (installs 713 skills
        to ~/.agent/skills)
Step 2. Open Antigravity IDE in Agent Mode on your CyberHawk project
        directory
Step 3. Paste each ANTIGRAVITY BUILD PROMPT block (marked in green)
        sequentially
Step 4. Each prompt references @skill-name tags that Antigravity will
        auto-invoke
Step 5. The agent will scaffold, code, test, and verify each component
        end-to-end
Step 6. UI components are pulled from 21st.dev using the @react-patterns +
        @typescript-expert skills

IMPORTANT: Run prompts in order.  Each phase builds on the previous one.
```

Contents

1 Master Build Strategy & Skills Map **4**
- 1.1 Project Identity . . . 4
- 1.2 Complete Skills Allocation Map . . . 4
- 1.3 Prompt Engineering Framework . . . 5

2 ANTIGRAVITY BUILD PROMPTS – Phase 1: Project Foundation **7**
- 2.1 Prompt AG-P1.1: Repository Analysis & Architecture Setup . . . 7
- 2.2 Prompt AG-P1.2: Database & Evidence Vault . . . 7
- 2.3 Prompt AG-P1.3: Security Hardening Layer . . . 8

3 ANTIGRAVITY BUILD PROMPTS – Phase 2: Device Detection Engine **9**
- 3.1 Prompt AG-P2.1: Hot-Plug Detection Service . . . 9
- 3.2 Prompt AG-P2.2: Evidence Acquisition Pipeline . . . 9

4 ANTIGRAVITY BUILD PROMPTS – Phase 3: Forensic Tool Engine **11**
- 4.1 Prompt AG-P3.1: Tool Adapter Framework . . . 11
- 4.2 Prompt AG-P3.2: YARA Rule Engine & Threat Intelligence . . . 12
- 4.3 Prompt AG-P3.3: LLM Orchestration Engine . . . 12

5 ANTIGRAVITY BUILD PROMPTS – Phase 4: Workflow Engine **14**
- 5.1 Prompt AG-P4.1: Orca Agentic Investigation Workflows . . . 14
- 5.2 Prompt AG-P4.2: FastAPI Backend Enhancement . . . 14

6 ANTIGRAVITY BUILD PROMPTS – Phase 5: 21st.dev UI Components **16**
- 6.1 21st.dev Component Specifications . . . 16
  - 6.1.1 Component 1: Halide Topo Hero . . . 16
  - 6.1.2 Component 2: Gooey Text Morphing . . . 16
  - 6.1.3 Component 3: Ultra Quality Showcase Grid . . . 17
  - 6.1.4 Component 4: Spatial Product Showcase . . . 17
- 6.2 Prompt AG-P5.1: Next.js Frontend Scaffold . . . 17
- 6.3 Prompt AG-P5.2: Core UI Views Implementation . . . 18
- 6.4 Prompt AG-P5.3: PyQt6 Desktop UI (Offline/Standalone Mode) . . . 19

7 ANTIGRAVITY BUILD PROMPTS – Phase 6: Voice, RAG & Reports **20**
- 7.1 Prompt AG-P6.1: Voice Interface (Whisper) . . . 20
- 7.2 Prompt AG-P6.2: Offline RAG Corpus Population . . . 20
- 7.3 Prompt AG-P6.3: Report Generation Engine . . . 21

8 ANTIGRAVITY BUILD PROMPTS – Phase 7: Integration & Testing 22
8.1 Prompt AG-P7.1: Full End-to-End Integration Test 22
8.2 Prompt AG-P7.2: Deployment Packaging 22

9 Complete File Structure 24
9.1 Target Directory Layout 24

10 Technical Specifications 26
10.1 Full Technology Stack 26
10.2 Performance Targets 27
10.3 Security Requirements Matrix 27

11 Non-Functional Requirements & Evaluation Criteria 28
11.1 Hackathon Evaluation Alignment 28

12 Quick-Reference Antigravity Prompt Summary 30
12.1 Complete Build Sequence 30
12.2 Final Smoke Test Prompt 31

13 Appendices 32
13.1 Appendix A: antigravity-awesome-skills Bundle Recommendation 32
13.2 Appendix B: System Prompt Design Patterns Used 32
13.3 Appendix C: 21st.dev Component Installation Commands 33
13.4 Appendix D: Python Requirements 33
13.5 Appendix E: Hackathon Demo Script (5 Minutes) 34

1. Master Build Strategy & Skills Map

1.1 Project Identity

Project Specification

Name	CyberHawk – Offline AI Forensics Platform
Codename	servos-cyberhawk
Stack	Python 3.11 + FastAPI + PyQt6 + Next.js 14 (Tauri) + Ollama + SQLite
LLM Engine	Ollama (llama3.1:8b, mistral:7b, phi3:mini, deepseek-r1:14b)
Skills Engine	antigravity-awesome-skills v4.11.0 (713 skills)
UI System	21st.dev components + Tailwind CSS + shadcn/ui
Prompt Ref	x1xhlol/system-prompts-and-models-of-ai-tools
Target ENV	100% Offline, Air-Gapped, Linux/Windows/macOS
Base Repo	github.com/Sansyuh06/servos (extend, don't rewrite)

1.2 Complete Skills Allocation Map

The following table maps every major CyberHawk module to the antigravity-awesome-skills that Antigravity will invoke when building it:

Skill	Module	What It Builds
`@security-auditing`	Core Security	Evidence integrity, audit logs, write-block
`@vulnerability-scanner`	Threat Detection	BadUSB DB, YARA engine, PE analysis
`@api-security-best-practices`	FastAPI Backend	Secure endpoints, JWT, rate limiting
`@python-patterns`	Forensic Engine	Tool adapters, async pipeline, evidence vault
`@react-patterns`	Frontend UI	Dashboard, workbench, case manager views
`@typescript-expert`	UI Layer	Type-safe Next.js + Tauri frontend
`@rag-engineer`	LLM Pipeline	ChromaDB, embeddings, offline RAG corpus
`@docker-expert`	Deployment	Offline Docker bundle, air-gapped packaging
`@test-driven-development`	Quality	Unit tests, integration tests, LLM eval suite

Skill	Module	What It Builds
`@brainstorming`	Architecture	Component design decisions, trade-off analysis
`@senior-architect`	System Design	7-layer architecture, data flow, API contracts
`@sql-injection-testing`	DB Security	SQLite hardening, parameterised queries
`@prompt-engineer`	LLM Prompts	Forensic system prompts, chain-of-thought
`@langgraph`	Agentic Flows	Orca workflow engine, investigation pipelines
`@workflow-automation`	Playbooks	USB triage, memory analysis, network IR
`@vercel-deployment`	Build System	Tauri bundle, offline installer packaging
`@writing-plans`	Documentation	README, API docs, evidence report templates

1.3 Prompt Engineering Framework

Drawing from the **x1xhlol/system-prompts-and-models-of-ai-tools** repository (123k stars), CyberHawk's Antigravity build uses the following patterns observed in production AI tools:

Pattern 1: Cursor-Style Codebase Context (from Cursor system prompt)

All build prompts inject full project context before requests. Antigravity is instructed to: read the existing `servos` repo structure first, understand existing `USBMonitor`, `InvestigationWorkflow`, and `FastAPI` code before writing new code. Never rewrite working code; extend it.

Pattern 2: Lovable-Style UI Component Scaffolding (from Lovable system prompt)

UI requests follow the Lovable pattern: specify exact component from 21st.dev by URL, describe exact props and data bindings, specify Tailwind classes with dark theme (`bg-slate-950`), and request responsive behaviour. No placeholder UI – real data connections required.

Pattern 3: Devin AI-Style Task Decomposition (from Devin system prompt)

Each build prompt is self-contained and ends with a verification command. Antigravity must confirm: (a) file created, (b) imports resolved, (c) tests passing, (d) no network calls in code. If any check fails, the agent retries automatically.

Pattern 4: v0-Style Component Generation (from v0 system prompt)

For all React/Next.js components, use shadcn/ui primitives as base, layer 21st.dev components on top, apply the project's dark cyber theme (`agdark`/`agprimary` palette). Every component ships with TypeScript types, JSDoc comments, and a Storybook story.

Pattern 5: Windsurf-Style Cascade Awareness

When building interdependent modules (e.g., USB Monitor → Evidence Vault → LLM), build bottom-up. Complete and test each layer before building the next. Use the `@senior-architect` skill to validate dependency graph before each phase.

2. ANTIGRAVITY BUILD PROMPTS – Phase 1: Project Foundation

PRE-FLIGHT CHECKLIST

Before running Phase 1 prompts, ensure: (1) Antigravity is open in the servos project root, (2) `npx antigravity-awesome-skills` has been run, (3) Ollama is installed with `llama3.1:8b` pulled, (4) Python 3.11+ is available, (5) Node.js 20+ is installed.

2.1 Prompt AG-P1.1: Repository Analysis & Architecture Setup

PASTE INTO ANTIGRAVITY – AG-P1.1

```
Use @senior-architect and @brainstorming skills.
Analyze the existing servos repository at the current directory.  Read
ALL existing files including:  - servos/detection/usb_monitor.py
- servos/orchestration/workflow.py - servos/api/server.py -
servos/database/db.py - src/ frontend components - Any existing README or
documentation
Then:  1.  Create a ARCHITECTURE.md documenting what already exists (do NOT
modify existing code yet) 2.  Create a PROJECT_GAPS.md listing what needs to
be built based on this PRD 3.  Create a DEPENDENCY_GRAPH.md showing module
dependencies 4.  Create a cyberhawk/config.py with ALL project constants
(model names, paths, thresholds) 5.  Create a cyberhawk/types.py with all
Pydantic v2 models for:  Device, EvidenceItem, Case, Finding, AuditLogEntry,
LLMResponse, ForensicToolResult 6.  Create a cyberhawk/constants.py with YARA
rule paths, tool binary paths, hash algorithms list
After creating files, run:  python -m py_compile cyberhawk/config.py
cyberhawk/types.py cyberhawk/constants.py
Verify:  Zero import errors.  Zero network imports.  All type hints present.
```

2.2 Prompt AG-P1.2: Database & Evidence Vault

PASTE INTO ANTIGRAVITY – AG-P1.2

```
Use @python-patterns and @sql-injection-testing skills.
Extend the existing servos/database/db.py to implement the full CyberHawk
schema:
CREATE these SQLite tables (append-only where specified):  - cases:  case_id,
case_number, investigator, mode, status, law_ref, created_at, closed_at
- evidence:  evidence_id, case_id, device_path, device_type, vendor_id,
sha256_pre, sha256_post, sha3_512, acquired_at, image_path, write_blocked
- audit_log (APPEND-ONLY): log_id AUTOINCREMENT, case_id, timestamp,
operator, action, tool, params_json, result_json, evidence_hash, hmac_sig
```

```
- conversations: msg_id, case_id, role, content, model_used, confidence,
timestamp, citations_json - findings: finding_id, case_id, severity, title,
description, evidence_refs, mitre_tactic, mitre_tech, confidence, created_at,
validated
IMPLEMENT cyberhawk/vault.py: - EvidenceVault class with AES-256-GCM
encryption (cryptography library) - PBKDF2 key derivation (100000 iterations,
SHA-256) from operator passphrase - store_evidence(path, passphrase) ->
EncryptedEvidenceRecord - retrieve_evidence(record_id, passphrase) -> bytes -
verify_integrity(record_id) -> bool (checks SHA-256 chain) - All operations
logged to audit_log with HMAC-SHA256 signature
Run: python -m pytest tests/test_vault.py -v (create the tests too) Verify:
All 8 tests pass. Zero plaintext key writes. All DB writes parameterised.
```

2.3 Prompt AG-P1.3: Security Hardening Layer

PASTE INTO ANTIGRAVITY – AG-P1.3

```
Use @security-auditing, @api-security-best-practices, and
@vulnerability-scanner skills.
Create cyberhawk/security/:
1. write_blocker.py: - LinuxWriteBlocker: uses hdparm -r1,
verifies with hdparm -I - WindowsWriteBlocker: registry
SYSTEM\CurrentControlSet\Control\StorageDevicePolicies WriteProtect=1
- MockWriteBlocker: for testing - All raise WriteBlockFailedException
if unsuccessful - Verify write-block before EVERY evidence operation via
decorator @require_write_block
2. network_sentinel.py: - Monitors ALL network interfaces using psutil -
Raises NetworkEgressAlertException on any non-loopback connection - Runs as
background thread during investigation sessions - Logs alert to audit_log and
sends toast notification
3. audit_integrity.py: - verify_log_chain(case_id): re-computes HMAC
for every audit entry, returns True/False - detect_tampering(): scans all
cases for broken HMAC chains - generate_integrity_report(): PDF report of
verification results
4. sandbox.py: - ForensicSandbox context manager using subprocess with: -
No network access (iptables drop or namespace isolation on Linux) - Read-only
evidence mount - Temp output directory cleaned after use - 5-minute timeout
with SIGKILL fallback
Run: python -m pytest tests/test_security.py -v Verify: All security tests
pass. No module imports socket or requests at module level.
```

3. ANTIGRAVITY BUILD PROMPTS – Phase 2: Device Detection Engine

3.1 Prompt AG-P2.1: Hot-Plug Detection Service

PASTE INTO ANTIGRAVITY – AG-P2.1

```
Use @python-patterns and @workflow-automation skills.
Extend servos/detection/usb_monitor.py into a full DeviceDetectionEngine:
CREATE cyberhawk/detection/device_engine.py:
class DeviceEvent(Enum): CONNECTED, DISCONNECTED, MODIFIED
@dataclass class DeviceProfile: device_id: str path: str vendor_id: str
product_id: str serial_number: str capacity_bytes: int filesystem_type:
str device_class: str % storage|hid|network|camera|unknown threat_match:
Optional[ThreatMatch] timestamp: datetime
class DeviceDetectionEngine: - Linux: pyudev UdevMonitor watching
subsystem=block + usb - Windows: win32api/WMI Win32_VolumeChangeEvent
watcher - macOS: plistlib parsing IOKit USBDevice events -
start_monitoring() -> runs background thread - on_device_connected(callback:
Callable[[DeviceProfile], None]) - on_device_disconnected(callback:
Callable[[DeviceProfile], None]) - get_connected_devices() ->
List[DeviceProfile]
CREATE cyberhawk/detection/threat_matcher.py: - Load offline BadUSB
signature DB from data/badusb_signatures.json (pre-populate with 50+ known
Rubber Ducky, Bash Bunny, OMG Cable VID/PIDs) - match_device(vid, pid)
-> Optional[ThreatMatch] - ThreatMatch includes: threat_name, severity,
description, recommended_action
CREATE cyberhawk/detection/device_profiler.py: - profile_device(path) ->
DeviceProfile - Uses: lsblk, fdisk -l, blkid, file -s (Linux) / diskpart
(Windows) - Determines filesystem, capacity, partition table - Extracts
S.M.A.R.T. data if available (smartmontools)
Run: python -m pytest tests/test_detection.py -v Verify: Mock USB insertion
triggers callback within 100ms in tests.
```

3.2 Prompt AG-P2.2: Evidence Acquisition Pipeline

PASTE INTO ANTIGRAVITY – AG-P2.2

```
Use @python-patterns and @security-auditing skills.
CREATE cyberhawk/acquisition/:
1. imager.py - ForensicImager class: - dd_image(source, dest,
block_size=512) -> ImageResult with SHA-256 - dcfldd_image(source, dest) ->
ImageResult (uses dcfldd if available, falls back to dd) - ewf_image(source,
dest) -> ImageResult (E01 via python-ewf or ewfacquire subprocess) -
```

```
verify_image(image_path, original_hash) -> bool - All operations:  pre-hash
source, acquire, post-hash both - abort if mismatch - Progress callback every
1% completion for UI updates
2.  memory_dumper.py - MemoryAcquirer class:  - linux_dump(output_path)
-> runs LiME via insmod, captures /proc/kcore - windows_dump(output_path)
-> runs winpmem.exe subprocess - verify_dump(path) -> checks file size
> 100MB (sanity), computes SHA-256 - is_lime_available() -> bool -
is_winpmem_available() -> bool
3.  backup_enforcer.py - BackupEnforcer class:  - This is the MANDATORY
backup gate - no investigation proceeds without passing this -
enforce(source_device, investigation_mode) -> BackupConfirmation - Creates
rsync/robocopy backup to investigator-specified destination - Verifies
backup via SHA-256 tree hash comparison - Returns BackupConfirmation with:
backup_path, verification_hash, timestamp - In Full Auto mode:  auto-selects
first available external drive if present - In Hybrid/Manual:  prompts
investigator via pyautogui dialog for destination
Run:  python -m pytest tests/test_acquisition.py -v Verify:  Imaging test
on a 50MB test image produces identical SHA-256.  Backup gate blocks if hash
mismatch.
```

4. ANTIGRAVITY BUILD PROMPTS – Phase 3: Forensic Tool Engine

4.1 Prompt AG-P3.1: Tool Adapter Framework

PASTE INTO ANTIGRAVITY – AG-P3.1

```
Use @python-patterns, @security-auditing, @test-driven-development skills.
CREATE cyberhawk/tools/base.py - ForensicToolAdapter ABC:
class ForensicToolAdapter(ABC): tool_id: str (abstract property)
supported_evidence_types: List[str] (abstract property) def
validate_input(self, evidence_path: Path) -> ValidationResult async
def execute(self, evidence_path, params, progress_cb) -> ToolResult
def to_llm_context(self, result: ToolResult) -> str (max 4096
tokens) def is_available(self) -> bool (checks binary/module exists)
requires_write_block: bool = True
ToolResult dataclass: tool_id, evidence_path, status
(success|error|timeout), findings: List[Finding], raw_output: str,
execution_time_ms: int, evidence_hash: str, timestamp: datetime
CREATE these 10 adapters in cyberhawk/tools/adapters/: 1. yara_adapter.py -
scans with python-yara, loads rules from data/yara_rules/ 2. tsk_adapter.py
- pytsk3 file system enumeration (all files + deleted + hidden) 3.
volatility_adapter.py - Volatility 3 subprocess for pslist, netscan, malfind,
cmdline, dlllist 4. exiftool_adapter.py - metadata extraction via exiftool
subprocess 5. strings_adapter.py - GNU strings + LLM analysis of suspicious
strings 6. registry_adapter.py - python-registry for Windows hive analysis
7. pefile_adapter.py - pefile library for PE header analysis + import
analysis 8. bulk_extractor_adapter.py - bulk_extractor subprocess for data
carving 9. tshark_adapter.py - tshark subprocess for PCAP analysis 10.
hashdeep_adapter.py - recursive SHA-256 + MD5 hash audit
CREATE cyberhawk/tools/tool_registry.py: - ToolRegistry: discovers
all adapters, checks availability, provides by evidence type -
get_tools_for_evidence(evidence_type) -> List[ForensicToolAdapter] -
run_all_applicable(evidence_path, case_id) -> List[ToolResult] (parallel
with asyncio)
Run: python -m pytest tests/test_tools/ -v Verify: All 10 adapters
instantiate. At least 5 adapters pass availability check (install missing
ones). YARA adapter scans test executable with known test rule.
```

4.2 Prompt AG-P3.2: YARA Rule Engine & Threat Intelligence

PASTE INTO ANTIGRAVITY – AG-P3.2

```
Use @vulnerability-scanner and @python-patterns skills.
CREATE data/yara_rules/ directory structure:  - malware/ (APT families,
ransomware, trojans) - exploits/ (CVE-specific patterns) - network/ (C2
communication patterns) - documents/ (malicious macro patterns, PDF exploits)
- living_off_the_land/ (LOLBin patterns) - usb_specific/ (USB-delivered
malware patterns) - custom/ (CyberHawk custom rules)
CREATE data/yara_rules/custom/cyberhawk.yar with 10+ custom rules:  -
USB-delivered payloads (autorun.inf patterns) - PowerShell encoded command
patterns - Common RAT strings (njRAT, DarkComet, AsyncRAT) - Keylogger API
calls pattern - Data exfiltration patterns (base64 + network functions) -
Anti-forensics patterns (log deletion, shadow copy deletion)
CREATE cyberhawk/threat_intel/yara_engine.py:  - YARAEngine:  loads ALL rules
from data/yara_rules/ recursively - scan_file(path) -> List[YARAMatch] with
rule_name, meta, strings_matched - scan_directory(path, recursive=True) ->
Dict[str, List[YARAMatch]] - explain_match(match, llm_client) -> str (LLM
explains what the rule means) - update_rules(rules_dir) -> int (count of
rules loaded) - get_statistics() -> dict (total rules, categories, last
updated)
CREATE cyberhawk/threat_intel/hash_lookup.py:  - OfflineHashDatabase:
loads pre-built SQLite DB of known malware hashes - lookup(sha256) ->
Optional[MalwareInfo] (name, family, threat level) - bulk_lookup(hashes:
List[str]) -> Dict[str, Optional[MalwareInfo]] - Create a seed database with
1000 test hashes using EICAR test file variants
Run:  python -m pytest tests/test_threat_intel.py -v Verify:  YARA engine
loads >5 rules.  EICAR test file matches test rule.  Hash lookup returns
result for EICAR SHA-256.
```

4.3 Prompt AG-P3.3: LLM Orchestration Engine

PASTE INTO ANTIGRAVITY – AG-P3.3

```
Use @rag-engineer, @prompt-engineer, and @langgraph skills.
CREATE cyberhawk/llm/:
1.  client.py - OllamaClient:  - ZERO external API calls.  All requests to
localhost:11434 ONLY - generate(model, prompt, system, temperature=0.1)
-> str - stream(model, prompt, system) -> AsyncGenerator[str, None] -
list_models() -> List[str] - is_model_available(model_name) -> bool - Raises
OfflineModeViolationException if connection to non-localhost detected
2.  router.py - ContextRouter:  TaskType enum:  LOG_ANALYSIS,
MALWARE_ANALYSIS, LEGAL_REFERENCE, MEMORY_FORENSICS, TIMELINE_NARRATE,
EVIDENCE_SUMMARY, VOICE_COMMAND, REPORT_WRITING, YARA_EXPLAIN, GENERAL_QUERY
TASK_MODEL_MAP =  TaskType.LOG_ANALYSIS: "mistral:7b",
TaskType.MALWARE_ANALYSIS: "qwen2.5:7b", TaskType.LEGAL_REFERENCE:
"deepseek-r1:14b", TaskType.MEMORY_FORENSICS: "llama3.1:8b",
TaskType.TIMELINE_NARRATE: "llama3.1:8b", TaskType.REPORT_WRITING:
"llama3.1:8b", TaskType.YARA_EXPLAIN: "qwen2.5:7b", TaskType.GENERAL_QUERY:
"llama3.1:8b",  route(task_type, available_vram_gb) -> ModelConfig with
```

```
fallback to phi3:mini
3.  prompts.py - FORENSIC_SYSTEM_PROMPTS dict:  One system prompt per
TaskType.  Each prompt includes:  - Role definition as DFIR expert -
Mandatory:  cite evidence by file path / line number / hash - Mandatory:
state [HIGH/MEDIUM/LOW] confidence - Mandatory:  say "insufficient evidence"
not speculate - Mandatory:  flag legal implications with applicable
law - Context injection template:  {case_context}, {evidence_summary},
{findings_so_far}
4.  rag.py - ForensicRAG: - Uses ChromaDB (embedded, no server) - Embedding:
sentence-transformers/all-MiniLM-L6-v2 (must be cached offline) - Corpus:
load from data/rag_corpus/ (NIST SP 800-86 chunks, MITRE ATT&CK, IT Act) -
query(question, top_k=5) -> List[RAGResult] with source, chunk, similarity -
augment_prompt(query, system_prompt) -> augmented prompt string
Run:  python -m pytest tests/test_llm.py -v Verify:  OllamaClient raises
exception if non-localhost URL used.  Router returns phi3:mini for 3GB VRAM.
RAG returns results for "chain of custody" query.
```

5. ANTIGRAVITY BUILD PROMPTS – Phase 4: Workflow Engine

5.1 Prompt AG-P4.1: Orca Agentic Investigation Workflows

PASTE INTO ANTIGRAVITY – AG-P4.1

```
Use @langgraph, @workflow-automation, and @python-patterns skills.
Extend servos/orchestration/workflow.py into a full Orca-based workflow
engine:
CREATE cyberhawk/workflow/:
1. planner.py - InvestigationPlanner: - Based on Orca agent architecture
from github.com/Sansyuh06/Orca - plan_investigation(device_profile, mode) ->
InvestigationPlan - InvestigationPlan: ordered list of InvestigationStep
with: tool, params, dependencies - Auto-selects tools based on device
type and evidence found - In Hybrid mode: inserts HumanCheckpoint steps
at critical junctures
2. executor.py - WorkflowExecutor (async): - execute_plan(plan, case_id,
progress_callback) -> WorkflowResult - Handles tool failures gracefully
(skip + log, don't abort) - Parallel execution of independent steps
(asyncio.gather) - Checkpoint saves state to SQLite every 60 seconds -
resume_from_checkpoint(case_id) -> WorkflowResult
3. playbooks.py - 4 pre-built playbooks as YAML: PLAYBOOK_USB_TRIAGE: steps
for USB device full analysis PLAYBOOK_MEMORY_ANALYSIS: LiME + Volatility
3 full suite PLAYBOOK_NETWORK_IR: PCAP analysis + tshark + LLM summary
PLAYBOOK_LOG_ANALYSIS: EVTX + syslog + auth.log parsing
4. human_loop.py - HumanInTheLoop: - present_decision(question, options,
context) -> str (selected option) - Full Auto mode: auto-selects recommended
option, logs the auto-decision - Hybrid mode: shows pyautogui dialog with
context, waits for operator input - Manual mode: shows dialog, waits with 0
default - All decisions logged to audit_log with timestamp and operator ID
Run: python -m pytest tests/test_workflow.py -v Verify: USB triage playbook
runs on test evidence directory. All 8 steps execute. Checkpoint saves
after step 3. Resume works from checkpoint.
```

5.2 Prompt AG-P4.2: FastAPI Backend Enhancement

PASTE INTO ANTIGRAVITY – AG-P4.2

```
Use @api-security-best-practices, @python-patterns, and
@test-driven-development skills.
Extend servos/api/server.py into the full CyberHawk FastAPI backend:
REQUIREMENTS: - All sockets bind to 127.0.0.1:8747 ONLY (never 0.0.0.0) -
JWT authentication for all endpoints (local secret, no external auth) - Rate
```

```
limiting:  100 req/min per endpoint - CORS: allow only tauri://localhost
and http://localhost:3000 - All responses use Pydantic v2 models from
cyberhawk/types.py
IMPLEMENT these endpoint groups:
/api/v1/devices:  GET / - list connected devices GET /{id} - device profile +
threat match POST /{id}/block - enforce write-block POST /{id}/image - start
forensic imaging (returns task_id)
/api/v1/cases:  GET / - list all cases with pagination POST / - create new
case GET /{id} - case details PUT /{id} - update status POST /{id}/start
- start investigation workflow GET /{id}/findings - all findings with
filters POST /{id}/report - generate report (PDF/DOCX/JSON) WS /{id}/stream -
WebSocket for real-time progress
/api/v1/llm:  POST /query - single LLM query with forensic context WS /stream
- streaming LLM WebSocket GET /models - available Ollama models POST /switch
- switch active model POST /rag/search - RAG semantic search
/api/v1/tools:  POST /execute - run specific forensic tool GET /available -
list tools with availability status WS /tasks/{id}/stream - tool execution
progress WebSocket
IMPLEMENT cyberhawk/api/auth.py:  - generate_local_token(operator_id,
passphrase) -> JWT - verify_token(token) -> operator_id - LocalAuth
dependency for FastAPI
Run:  uvicorn cyberhawk.api.server:app -host 127.0.0.1 -port 8747 Run:
python -m pytest tests/test_api.py -v Verify:  All endpoints return 401
without JWT. WebSocket streams data.  Zero 0.0.0.0 bindings.
```

6. ANTIGRAVITY BUILD PROMPTS – Phase 5: 21st.dev UI Components

6.1 21st.dev Component Specifications

The following four components from 21st.dev form the visual identity of CyberHawk's frontend. Each has a specific role and integration specification.

6.1.1 Component 1: Halide Topo Hero

21st.dev: halide-topo-hero (by shivendra9795kumar) – CyberHawk Dashboard Hero

Source: `21st.dev/community/components/shivendra9795kumar/halide-topo-hero/default`
Effect: Animated topographic contour lines with halide colour shifting – perfect for a cyberpunk dashboard header
CyberHawk Application:

- Used as the **main dashboard hero section** behind the "Active Investigation" status card
- Colour scheme adapted: `hsl(195, 100%, 42%)` (cyan) contour lines on `#030712` (slate-950) background
- Animation speed reduced to 0.3x for "surveillance monitor" feel during active investigations
- Hero text overlay: "CYBERHAWK" in Geist Mono with real-time threat count badge
- When BadUSB is detected, animation pulses red (`hsl(0, 84%, 60%)`)

6.1.2 Component 2: Gooey Text Morphing

21st.dev: gooey-text-morphing (by victorwelander) – AI Status Display

Source: `21st.dev/community/components/victorwelander/gooey-text-morphing/default`
Effect: Smooth gooey morphing between text strings using SVG filter blur + contrast trick
CyberHawk Application:

- Used in the **AI Assistant panel header** to cycle through LLM status messages
- Morphs between: "Analysing Evidence..." → "Cross-referencing YARA..." → "Querying ATT&CK..." → "Generating Report..."
- Active model name displayed below: `llama3.1:8b | mistral:7b | phi3:mini`
- Text colour: `#06b6d4` (cyan-500) matching the CyberHawk palette
- Font: Geist Mono – reinforces terminal/forensic aesthetic

6.1.3 Component 3: Ultra Quality Showcase Grid

21st.dev: ultra-quality-showcase-grid (by dhileepkumargm) – Forensic Tools Grid

Source: `21st.dev/community/components/dhileepkumargm/ultra-quality-showcase-grid/def`
Effect: High-quality bento grid with hover depth effects, gradient borders, and animated reveal
CyberHawk Application:

- Used as the **"Available Forensic Tools" grid** on the Investigation Workbench page
- 25 tool cards in a responsive masonry grid
- Each card: tool name, category badge (Disk/Memory/Network/Malware), status dot (green=available, red=missing), last-run timestamp
- Hover reveals: tool description, supported evidence types, "Run Now" CTA button
- Clicking a card opens a side-panel with tool parameters form + "Execute" button
- Data pulled from `GET /api/v1/tools/available` endpoint

6.1.4 Component 4: Spatial Product Showcase

21st.dev: spatial-product-showcase (by daiv09) – Evidence 3D Viewer

Source: `21st.dev/community/components/daiv09/spatial-product-showcase/default`
Effect: 3D spatial parallax product showcase with depth layers and mouse-reactive movement
CyberHawk Application:

- Used on the **Evidence Vault page** to showcase active investigation summary
- Three depth layers: Background (case metadata), Mid (evidence items grid), Foreground (critical findings)
- Mouse parallax effect creates a "looking into the evidence" immersive feel
- Used specifically for the **Case Summary hero card** when viewing a specific case
- Data: case number, investigator, device info, top 3 critical findings floating in parallax
- Clicking a finding navigates to the full finding detail in the Workbench

6.2 Prompt AG-P5.1: Next.js Frontend Scaffold

PASTE INTO ANTIGRAVITY – AG-P5.1

```
Use @react-patterns, @typescript-expert, and @vercel-deployment skills.
Scaffold the CyberHawk Next.js 14 frontend in the frontend/ directory:
npx create-next-app@14 frontend -typescript -tailwind -app -src-dir
-import-alias "@/*"
Install dependencies:  npm install @radix-ui/react-* shadcn-ui lucide-react
framer-motion npm install @tanstack/react-query axios zod react-hook-form npm
install geist next-themes class-variance-authority clsx tailwind-merge
IMPLEMENT Tailwind config extending theme with CyberHawk palette:  - agdark:
'#030712', agmid:  '#0f172a', agsurface:  '#1e293b' - agborder:  '#334155',
agprimary:  '#06b6d4', aggreen:  '#22c55e' - agred:  '#ef4444', agyellow:
```

```
'#eab308', agpurple: '#a855f7' - Font: Geist Mono for all mono elements,
Geist Sans for body
IMPLEMENT these TypeScript types in frontend/src/types/: - device.ts:
Device, DeviceProfile, ThreatMatch, WriteBlockStatus - case.ts: Case,
CaseStatus, InvestigationMode, AuditLogEntry - finding.ts: Finding,
FindingSeverity, MITRETechnique - llm.ts: LLMMessage, LLMResponse,
ModelConfig, RAGResult - tool.ts: ForensicTool, ToolResult, ToolStatus
IMPLEMENT frontend/src/lib/: - api.ts: Axios instance pointing to
http://127.0.0.1:8747, JWT header injection - websocket.ts: WebSocketManager
for investigation streaming + LLM streaming - utils.ts: cn(), formatBytes(),
formatDuration(), hashDisplay() - constants.ts: SEVERITY_COLORS,
TOOL_CATEGORIES, MITRE_TACTICS
Run: npm run build Verify: Zero TypeScript errors. Build succeeds. All
imports resolve.
```

6.3 Prompt AG-P5.2: Core UI Views Implementation

PASTE INTO ANTIGRAVITY – AG-P5.2

```
Use @react-patterns, @typescript-expert skills.
IMPORTANT: Copy the 21st.dev components by installing
them via their registry commands: npx shadcn@latest add
"https://21st.dev/r/shivendra9795kumar/halide-topo-hero" npx shadcn@latest
add "https://21st.dev/r/victorwelander/gooey-text-morphing" npx shadcn@latest
add "https://21st.dev/r/dhileepkumargm/ultra-quality-showcase-grid" npx
shadcn@latest add "https://21st.dev/r/daiv09/spatial-product-showcase"
BUILD these 7 page views in frontend/src/app/:
PAGE 1: /dashboard (page.tsx) - HalideTopo hero component behind stats
header - GooeyTextMorphing in LLM status area cycling investigation states
- DeviceMonitorCard: live list from GET /api/v1/devices, 2s polling -
ActiveInvestigationCard: progress bar + WebSocket stream for active
case - SystemHealthCard: CPU/RAM/GPU via internal metrics endpoint -
RecentCasesCard: last 5 cases from GET /api/v1/cases?limit=5
PAGE 2: /workbench/[caseId] (page.tsx) - UltraQualityShowcaseGrid for
25 forensic tools - TerminalOutput component: real-time tool output via
WebSocket - TimelineViewer: horizontal scrollable forensic timeline -
FindingsPanel: sortable/filterable findings list with severity badges -
SplitPane layout: tools left 40%, output right 60%
PAGE 3: /vault (page.tsx) - SpatialProductShowcase for active case summary
hero - EvidenceTable: DataTable with sorting, filtering, hash verification
status - ChainOfCustodyViewer: visual step-by-step custody chain with hash
verifications - EncryptionStatus: AES-256 status indicator for each evidence
item
PAGE 4: /assistant (page.tsx) - GooeyTextMorphing in header cycling
model names - Chat interface: messages with role, confidence badges,
evidence citations - ModelSelector: dropdown for switching Ollama models
- QuickQueryButtons: pre-built forensic query templates - VoiceInputButton:
records + sends to /api/v1/llm/voice
```

```
PAGE 5: /playbooks (page.tsx) - PlaybookCard grid: 4 playbooks with
animated progress rings - PlaybookDetail modal: step-by-step view with
current step highlighted
PAGE 6: /cases (page.tsx) - Full DataTable with pagination, status filters,
search - NewCaseModal: form with case number, investigator, mode selector
PAGE 7: /settings (page.tsx) - ModelConfigPanel: Ollama model manager
- ToolPathsPanel: verify tool binary paths - SecurityPanel: passphrase
change, write-block test
Run: npm run dev Verify: All 7 pages render without console errors. API
connection established.
```

6.4 Prompt AG-P5.3: PyQt6 Desktop UI (Offline/Standalone Mode)

PASTE INTO ANTIGRAVITY – AG-P5.3

```
Use @python-patterns and @react-patterns skills.
BUILD cyberhawk/ui/pyqt/ - the offline PyQt6 desktop interface (when no
browser available):
IMPLEMENT these QWidget subclasses with cyberpunk dark theme (bg: #030712,
accent: #06b6d4):
1. MainWindow: 7-tab interface matching the web views - NavigationRail:
left sidebar with icon + label tabs, active state glow effect - StatusBar:
LLM model indicator, write-block status LED, network sentinel indicator -
ToastManager: stacked notifications top-right, auto-dismiss after 5s -
Keyboard shortcuts: Ctrl+1-7 for tab switching, Ctrl+N new case, Ctrl+R
report
2. DashboardWidget: device cards + active investigation card -
USBAlertWidget: pops up automatically on device insertion (always-on-top)
- Shows device info, threat status, action buttons (Block / Investigate /
Dismiss)
3. WorkbenchWidget: - TerminalWidget: QPlainTextEdit with ANSI
colour support (green for success, red for error) - TimelineWidget:
QGraphicsScene-based scrollable timeline - FindingsTreeWidget: QTreeWidget
with severity icons and MITRE tags
4. AssistantWidget: - ChatHistoryWidget: message bubbles with role colours
- InputBar: QLineEdit + Send button + Voice button - ModelStatusLabel:
GooeyTextMorphing equivalent (QLabel with QPropertyAnimation)
Apply QSS stylesheet from cyberhawk/ui/styles/dark_cyber.qss: - All
backgrounds: #030712 to #1e293b range - Accent: #06b6d4 - Success:
#22c55e, Error: #ef4444, Warning: #eab308 - Font: monospace 12pt for
terminal, 11pt sans for UI
Run: python -m cyberhawk.ui.pyqt.main Verify: Window opens. All 7 tabs
navigate. Device list populates. LLM chat sends request.
```

7. ANTIGRAVITY BUILD PROMPTS – Phase 6: Voice, RAG & Reports

7.1 Prompt AG-P6.1: Voice Interface (Whisper)

PASTE INTO ANTIGRAVITY – AG-P6.1

```
Use @python-patterns skill.
CREATE cyberhawk/voice/:
1. transcriber.py - OfflineTranscriber: - Uses openai-whisper library
(LOCAL inference, NO API calls) - Model: whisper.load_model("base") -
74MB, runs on CPU - transcribe(audio_path) -> TranscriptionResult with text,
confidence, language - transcribe_stream(mic_stream) -> AsyncGenerator[str,
None] for live transcription - ASSERT: zero network connections during
transcription (test with mock socket)
2. tts.py - OfflineTTS: - Uses piper-tts (fastest offline TTS for
Linux/Windows) - Falls back to espeak-ng if piper not available - speak(text:
str) -> plays audio via sounddevice - speak_async(text: str) -> non-blocking
via threading.Thread - Voices: en_US-amy-medium (piper) or en+m1 (espeak)
3. voice_command_handler.py - VoiceCommandHandler: - Maps
transcribed commands to API actions: "start investigation" -> POST
/api/v1/cases/{id}/start "generate report" -> POST /api/v1/cases/{id}/report
"show findings" -> navigate to findings panel "what is [X]" -> send to LLM
assistant "stop" -> pause current workflow - Confidence threshold: 0.75
(ignores low-confidence transcriptions) - All commands logged to audit_log
Run: python -m pytest tests/test_voice.py -v Verify: Transcriber processes
test WAV file without network. TTS plays audio locally.
```

7.2 Prompt AG-P6.2: Offline RAG Corpus Population

PASTE INTO ANTIGRAVITY – AG-P6.2

```
Use @rag-engineer skill.
CREATE cyberhawk/rag/corpus_builder.py:
BUILD the offline RAG corpus from freely available sources that can be
bundled:
1. MITRE ATT&CK: Download enterprise-attack.json from official MITRE repo
- Extract all techniques with: id, name, description, tactics, examples -
Chunk each technique description into 512-token chunks with 64 overlap -
Metadata: technique_id, tactic, url
2. NIST SP 800-86 (forensics guide): Include as data/nist_800_86.txt -
Pre-chunk into 512-token sections - Metadata: section_number, title
3. ISO 27037 Summary: Include as data/iso_27037_summary.txt - Key
principles: identification, collection, acquisition, preservation
```

```
4. IT Act 2000 India: Include key sections as data/it_act_2000.txt -
Sections 43, 65, 65B, 66, 66B-F, 67, 72, 84A, 84B
5. GDPR Key Articles: data/gdpr_key_articles.txt - Articles 4, 5, 17, 25,
32, 33, 34, 83 with summaries
6. Common Forensic Procedures: data/forensic_procedures.txt -
Write-blocking procedures, chain of custody, evidence handling
IMPLEMENT build_corpus(): - Creates ChromaDB collection
"cyberhawk_legal_forensic" - Embeds all chunks using sentence-transformers
all-MiniLM-L6-v2 (offline) - Persists to data/chroma_db/ directory - Returns
corpus stats: total chunks, collections, size
IMPLEMENT query_corpus(question, top_k=5) -> List[RAGResult]: - Semantic
search across all collections - Returns: content, source, section,
similarity_score
Run: python cyberhawk/rag/corpus_builder.py -build Run: python -m pytest
tests/test_rag.py -v Verify: Corpus has >500 chunks. Query "chain of
custody" returns NIST and ISO results.
```

7.3 Prompt AG-P6.3: Report Generation Engine

PASTE INTO ANTIGRAVITY – AG-P6.3

```
Use @python-patterns, @writing-plans skills.
CREATE cyberhawk/reporting/:
1. pdf_reporter.py - PDFReporter (using ReportLab): -
generate_executive_report(case_id) -> Path (PDF file) - Cover page:
case number, investigator, dates, classification - Executive Summary:
LLM-generated 3-paragraph summary of investigation - Chain of Custody: table
of all evidence items with hash verifications - Timeline: visual forensic
timeline (matplotlib embedded as image) - Findings: each finding with
severity badge, evidence citations, MITRE ATT&CK mapping - Appendix: full
audit log, tool outputs - Footer: SHA-256 of entire report (self-verifying
document) - Use corporate dark theme: #030712 cover, #06b6d4 accent colours
2. docx_reporter.py - DOCXReporter (using python-docx): -
generate_technical_report(case_id) -> Path (DOCX file) - Structured for peer
review: full tool outputs, raw evidence references - Auto-generates Table of
Contents - Includes all 21st.dev aesthetic elements as header images
3. json_reporter.py - JSONReporter: - generate_machine_report(case_id)
-> Path (JSON file) - STIX 2.1 compatible format for SIEM/SOAR integration
- Include: all findings as STIX Observed Data objects - Include: threat
actors as STIX Threat Actor objects if identified
4. timeline_generator.py - TimelineGenerator: - generate_timeline(case_id)
-> matplotlib Figure - X-axis: time, Y-axis: event category - Events
colour-coded by severity - Exported as PNG for embedding in reports
Run: python -m pytest tests/test_reporting.py -v Verify: PDF generates
successfully for test case. Contains >5 pages. Chain of custody section
present. SHA-256 footer computed.
```

8. ANTIGRAVITY BUILD PROMPTS – Phase 7: Integration & Testing

8.1 Prompt AG-P7.1: Full End-to-End Integration Test

PASTE INTO ANTIGRAVITY – AG-P7.1

```
Use @test-driven-development and @security-auditing skills.
CREATE tests/integration/test_full_pipeline.py:
TEST SCENARIO 1: Full USB Triage (Full Auto Mode) 1. Create a synthetic
50MB test "USB image" with: - 100 normal files (txt, jpg, pdf) - 3
YARA-matching test malware samples (EICAR + custom rules) - 10 deleted
files (allocate + zero blocks) - 1 suspicious PE file with packed entropy
2. Simulate device connection via DeviceDetectionEngine mock 3. Run
BackupEnforcer (to temp directory) 4. Execute PlaybookUSBTriage in Full
Auto mode 5. Verify: backup created + verified; all files enumerated; YARA
matches found; deleted files recovered; LLM summary generated; PDF report
created 6. Verify: SHA-256 chain unbroken throughout; audit log has >20
entries; all audit entries have valid HMAC; zero network calls made
TEST SCENARIO 2: Chain of Custody Integrity 1. Run a mini investigation
2. Manually corrupt one audit log entry 3. Run verify_log_chain()
- must detect corruption and identify the entry 4. Verify:
audit_integrity.detect_tampering() returns True for corrupted case
TEST SCENARIO 3: LLM Hallucination Guard 1. Feed LLM tool outputs with
fabricated findings not in evidence 2. HallucinationGuard should flag
uncorroborated claims 3. Response must include "insufficient evidence" for
fabricated items
TEST SCENARIO 4: Network Egress Check 1. Start NetworkSentinel 2. Attempt
HTTP request from a test subprocess 3. Sentinel must log the attempt and
trigger alert within 500ms
Run: python -m pytest tests/integration/ -v -timeout=120 Verify: All 4
scenarios pass. Generate coverage report: >80% coverage overall.
```

8.2 Prompt AG-P7.2: Deployment Packaging

PASTE INTO ANTIGRAVITY – AG-P7.2

```
Use @docker-expert and @vercel-deployment skills.
CREATE deployment/:
1. Dockerfile.cyberhawk: - Base: python:3.11-slim-bookworm - Install: all
system forensic tools via apt-get (offline .deb packages in packages/) -
Install: all Python deps from requirements.txt (no pip install -index-url)
- Copy: all LLM models from models/ directory into /app/models/ - EXPOSE:
127.0.0.1:8747 ONLY (no external exposure) - ENV: CYBERHAWK_OFFLINE=1,
```

```
OLLAMA_HOST=http://localhost:11434 - HEALTHCHECK: python -c "import
cyberhawk; cyberhawk.health_check()"
2.  docker-compose.yml:  services:  ollama:  image ollama/ollama, volumes
models:/root/.ollama cyberhawk:  build Dockerfile.cyberhawk, depends_on
ollama frontend:  npm run build + serve on localhost:3000 networks:
cyberhawk-internal (internal:  true - NO external routing)
3.  install/offline_setup.sh:  - Full offline installer script - Checks:
Python 3.11, Node 20, Docker (optional) - Installs forensic tools from
packages/ directory - Pulls Ollama models from local bundle - Populates
ChromaDB corpus - Creates systemd service for auto-start - Runs smoke test:
python -m cyberhawk.smoke_test
4.  Makefile:  make dev - start all services in development mode make test
- run full test suite make build - build Tauri desktop app make package -
create offline deployment bundle make smoke - quick 2-minute end-to-end smoke
test
Run:  docker build -f deployment/Dockerfile.cyberhawk -t cyberhawk:latest
.  Verify:  Build succeeds.  Container starts.  Health check passes.  Zero
external network calls.
```

9. Complete File Structure

9.1 Target Directory Layout

After all Antigravity build prompts are executed, the repository should have this structure:

```
cyberhawk/                          # Root (extends servos/ repo)
|-- cyberhawk/                      # Python package
|    |-- __init__.py, config.py, types.py, constants.py
|    |-- acquisition/               # Backup, imaging, memory dump
|    |    |-- backup_enforcer.py, imager.py, memory_dumper.py
|    |-- api/                       # FastAPI backend
|    |    |-- server.py, auth.py, websockets.py, models.py
|    |-- detection/                 # Device hot-plug detection
|    |    |-- device_engine.py, device_profiler.py, threat_matcher.
     py
|    |-- llm/                       # Ollama LLM integration
|    |    |-- client.py, router.py, prompts.py, rag.py
|    |-- rag/                       # ChromaDB offline RAG
|    |    |-- corpus_builder.py, retriever.py
|    |-- reporting/                 # PDF/DOCX/JSON/STIX reports
|    |    |-- pdf_reporter.py, docx_reporter.py, json_reporter.py
|    |-- security/                  # Security hardening
|    |    |-- audit_integrity.py, network_sentinel.py
|    |    |-- sandbox.py, write_blocker.py
|    |-- threat_intel/              # YARA + hash DB
|    |    |-- yara_engine.py, hash_lookup.py
|    |-- tools/                     # Forensic tool adapters
|    |    |-- base.py, tool_registry.py
|    |    |-- adapters/ (10 adapter files)
|    |-- ui/                        # PyQt6 desktop UI
|    |    |-- pyqt/main.py, styles/dark_cyber.qss
|    |-- vault.py                   # AES-256 evidence vault
|    |-- voice/                     # Whisper + piper TTS
|    |    |-- transcriber.py, tts.py, voice_command_handler.py
|    |-- workflow/                  # Orca workflow engine
|         |-- executor.py, human_loop.py, planner.py, playbooks.py
|-- frontend/                       # Next.js 14 frontend
|    |-- src/app/ (7 pages)
|    |-- src/components/ (21st.dev + custom)
|    |-- src/lib/ (api, websocket, utils)
|    |-- src/types/ (TypeScript types)
|-- data/                           # Offline data bundles
|    |-- yara_rules/ (malware, exploits, custom)
|    |-- chroma_db/ (pre-built RAG corpus)
|    |-- badusb_signatures.json
|    |-- nist_800_86.txt, it_act_2000.txt, gdpr_key_articles.txt
|-- deployment/
```

```
|    |-- Dockerfile.cyberhawk, docker-compose.yml, offline_setup.sh
|-- tests/ (unit, integration, security tests)
|-- Makefile, pyproject.toml, package.json
```

10. Technical Specifications

10.1 Full Technology Stack

Layer	Technology	Purpose
LLM Runtime	Ollama 0.3+	Local model serving
LLM Models	Llama 3.1 8B Q4, Mistral 7B Q4, Phi-3 Mini, DeepSeek-R1 14B	Multi-model inference
LLM API	llama.cpp HTTP + Ollama REST	Offline inference API
RAG Store	ChromaDB (embedded)	Offline vector search
Embeddings	sentence-transformers all-MiniLM-L6-v2	Local embeddings
Backend	Python 3.11 + FastAPI	Core API server
DB	SQLite 3.40+ (WAL mode)	Persistent storage
Crypto	cryptography 41+ (AES-256-GCM)	Evidence encryption
Auth	python-jose (JWT local)	Local authentication
Async	asyncio + uvicorn	Async server runtime
Frontend (Web)	Next.js 14 + React 18	Web UI
Frontend (Desktop)	PyQt6 + QSS	Offline GUI
UI Components	21st.dev + shadcn/ui	Premium components
Styling	Tailwind CSS + CSS Variables	Design system
State	TanStack Query v5	Server state management
Forensic Tools	pytsk3, Volatility 3, YARA	Core analysis
Acquisition	dcfldd, dd, LiME, winpmem	Evidence acquisition
Network	tshark, tcpdump	Network analysis
Metadata	exiftool, pefile	Artifact extraction
Carving	foremost, bulk_extractor	Data recovery
Voice	openai-whisper (local), piper-tts	Offline voice I/O
Timeline	log2timeline/plaso	Forensic timeline
Packaging	Docker + Tauri 2.0	Deployment

10.2 Performance Targets

Metric	Target	Acceptable	Measurement
LLM First Token (GPU)	<2s	<4s	P95 latency
LLM First Token (CPU)	<8s	<15s	P95 latency
Device Detection	<2s	<5s	Insert to notification
USB Triage (32GB)	<8 min	<15 min	Full auto pipeline
Memory Acquisition	<3 min	<6 min	LiME on 8GB RAM
YARA Scan (1GB)	<45s	<90s	15,000 rules
RAG Query	<500ms	<1.5s	Top-5 retrieval
Report Generation	<30s	<60s	PDF + DOCX
App Cold Start	<5s	<10s	GUI ready

10.3 Security Requirements Matrix

Requirement	Priority	Phase	Implementation
Zero network egress	**HIGH**	P1	NetworkSentinel + loopback-only binding
AES-256 evidence	**HIGH**	P1	EvidenceVault with PBKDF2 keys
Write-block before access	**HIGH**	P1	@require_write_block decorator
HMAC audit log	**HIGH**	P1	SHA-256 HMAC on each log entry
JWT local auth	**HIGH**	P1	python-jose, local secret
Subprocess sandbox	**HIGH**	P1	ForensicSandbox context manager
Secure temp wipe	**MED**	P2	DoD 5220.22-M secure delete
Hardware token auth	**LOW**	P3	YubiKey HMAC-SHA1 challenge-response
Anti-forensics detection	**MED**	P2	Entropy analysis, log-clear detection

11. Non-Functional Requirements & Evaluation Criteria

11.1 Hackathon Evaluation Alignment

Evaluation Parameter 1: Offline Capability (20 pts)

Implementation Evidence:
- NetworkSentinel running throughout – alerts on any egress attempt
- All LLM inference via Ollama pointing to `localhost:11434`
- ChromaDB, YARA rules, hash DB all local files
- FastAPI bound to `127.0.0.1:8747` exclusively
- Docker network configured as `internal: true`
- Automated test: 1-hour network monitor during full investigation, zero connections

Demo Evidence: Show Wireshark capture during live demo – zero outbound packets.

Evaluation Parameter 2: Accuracy of Responses (20 pts)

Implementation Evidence:
- Multi-model routing selects best model per task type
- Evidence-citation mandatory in all LLM responses
- YARA validation cross-checks LLM malware claims
- RAG augments responses with NIST/ATT&CK citations
- Confidence levels (HIGH/MEDIUM/LOW) required in all forensic findings
- 50-question benchmark eval: target 92%+ factual accuracy

Evaluation Parameter 3: Security & Privacy (20 pts)

Implementation Evidence:
- AES-256-GCM evidence vault with PBKDF2 key derivation
- HMAC-SHA256 signed append-only audit log
- Write-blocker enforced before any evidence access
- Zero telemetry, zero external API calls, zero crash reporting
- Evidence hash chain: SHA-256 + SHA-3-512 on every operation
- BadUSB threat matching before any device is processed

Evaluation Parameter 4: Performance Efficiency (20 pts)

Implementation Evidence:
- Async FastAPI with parallel tool execution (asyncio.gather)
- Streaming LLM responses – first token in <2s on GPU
- YARA scanning in parallel across file chunks

- SQLite WAL mode for concurrent reads during analysis
- React Query caching eliminates redundant API calls
- Quantized Q4 models balance accuracy vs. inference speed

Evaluation Parameter 5: Practical Investigation Support (20 pts)

Implementation Evidence:

- 25 integrated forensic tools covering Disk/Memory/Network/Malware
- 4 pre-built investigation playbooks for common scenarios
- Voice interface for hands-free field operation
- Three investigation modes: Full Auto / Hybrid / Manual
- Court-admissible PDF + DOCX report generation
- Offline legal reference (IT Act, GDPR, NIST, ATT&CK) via RAG
- MITRE ATT&CK technique mapping for all findings

12. Quick-Reference Antigravity Prompt Summary

12.1 Complete Build Sequence

This chapter serves as the quick-reference checklist. Paste each prompt into Antigravity in order, verifying the "Check" condition before proceeding.

Prompt	What Gets Built	Skills Used	Verify
AG-P1.1	Architecture + Types + Constants	@senior-architect @brainstorming	py_compile passes
AG-P1.2	Database + Evidence Vault	@python-patterns @sql-injection-testing	8 vault tests pass
AG-P1.3	Security Hardening Layer	@security-auditing @vulnerability-scanner	Security tests pass
AG-P2.1	Device Detection Engine	@python-patterns @workflow-automation	Callback <100ms
AG-P2.2	Evidence Acquisition	@python-patterns @security-auditing	Hash chain verified
AG-P3.1	Tool Adapter Framework	@python-patterns @test-driven-development	10 adapters load
AG-P3.2	YARA + Threat Intel	@vulnerability-scanner	EICAR match found
AG-P3.3	LLM Orchestration	@rag-engineer @prompt-engineer @langgraph	RAG returns results
AG-P4.1	Orca Workflow Engine	@langgraph @workflow-automation	Playbook executes
AG-P4.2	FastAPI Backend	@api-security-best-practices	JWT auth works
AG-P5.1	Next.js Frontend Scaffold	@react-patterns @typescript-expert	Build succeeds
AG-P5.2	7 Web UI Views + 21st.dev	@react-patterns @typescript-expert	All pages render
AG-P5.3	PyQt6 Desktop UI	@python-patterns	Window opens
AG-P6.1	Voice Interface (Whisper)	@python-patterns	Transcription works

Prompt	What Gets Built	Skills Used	Verify
AG-P6.2	RAG Corpus Population	@rag-engineer	500+ chunks loaded
AG-P6.3	Report Generation	@python-patterns @writing-plans	PDF generates
AG-P7.1	Integration Tests	@test-driven-development	All 4 scenarios pass
AG-P7.2	Docker + Packaging	@docker-expert	Container starts

12.2 Final Smoke Test Prompt

FINAL ANTIGRAVITY VERIFICATION PROMPT

Use @test-driven-development and @security-auditing skills.
Run the complete CyberHawk smoke test:
1. python -m cyberhawk.smoke_test – runs 10 core function checks 2. npm run build (frontend) – must complete with zero errors 3. python -m pytest tests/ -v –timeout=60 – all tests must pass 4. python -m cyberhawk.network_audit – must report ZERO egress attempts 5. python -m cyberhawk.demo – runs the full USB triage demo with synthetic evidence
Generate a SMOKE_TEST_REPORT.md with: - Pass/fail for each component - LLM response sample (showing citation + confidence) - YARA match sample (showing rule name + file + strings) - Chain of custody hash chain (showing 5-step verification) - Performance numbers (LLM latency, scan speed, detection time)
This report is the final evidence for hackathon judges.

13. Appendices

13.1 Appendix A: antigravity-awesome-skills Bundle Recommendation

For CyberHawk, we recommend the **Security Engineer** + **Full-Stack Developer** bundle from the antigravity-awesome-skills repository. Install with:

```
# Install all skills
npx antigravity-awesome-skills

# Verify critical skills are present
ls ~/.agent/skills/security-auditing/
ls ~/.agent/skills/vulnerability-scanner/
ls ~/.agent/skills/rag-engineer/
ls ~/.agent/skills/python-patterns/
ls ~/.agent/skills/react-patterns/
ls ~/.agent/skills/typescript-expert/
ls ~/.agent/skills/docker-expert/
ls ~/.agent/skills/test-driven-development/
ls ~/.agent/skills/api-security-best-practices/
ls ~/.agent/skills/prompt-engineer/
ls ~/.agent/skills/langgraph/
echo "All required skills present!"
```

13.2 Appendix B: System Prompt Design Patterns Used

From the **x1xhlol/system-prompts-and-models-of-ai-tools** repository, these specific patterns were incorporated:

Tool Source	Pattern Borrowed	CyberHawk Application
Cursor	Full codebase context before edits	All AG prompts read existing servos code first
Lovable	Component-first UI with exact specs	21st.dev components specified by exact URL
Devin AI	Self-contained tasks with verification	Every prompt ends with a verify command
v0	shadcn/ui base + layer custom	shadcn primitives + 21st.dev on top
Windsurf	Cascade/bottom-up building	Phase ordering: security → backend → UI
Replit	Always-runnable code	Every prompt creates immediately runnable code

Tool Source	Pattern Borrowed	CyberHawk Application
Manus	Autonomous multi-step execution	Multi-file creation in single prompt
Claude Code	Tool use + file operations	Antigravity invokes @skill tools natively

13.3 Appendix C: 21st.dev Component Installation Commands

```
# Install all 4 CyberHawk UI components from 21st.dev
# (run from frontend/ directory after Next.js scaffold)

# 1. Halide Topo Hero -- Dashboard hero background
npx shadcn@latest add \
  "https://21st.dev/r/shivendra9795kumar/halide-topo-hero"

# 2. Gooey Text Morphing -- LLM status display
npx shadcn@latest add \
  "https://21st.dev/r/victorwelander/gooey-text-morphing"

# 3. Ultra Quality Showcase Grid -- Forensic tools grid
npx shadcn@latest add \
  "https://21st.dev/r/dhileepkumargm/ultra-quality-showcase-grid"

# 4. Spatial Product Showcase -- Evidence vault 3D view
npx shadcn@latest add \
  "https://21st.dev/r/daiv09/spatial-product-showcase"

# Verify all components installed
ls src/components/ui/ | grep -E "halide|gooey|showcase|spatial"
```

13.4 Appendix D: Python Requirements

```
# requirements.txt -- ALL PACKAGES MUST BE INSTALLABLE OFFLINE
# (pre-download wheels to packages/wheels/ for air-gapped install)

# Core
fastapi==0.115.0
uvicorn[standard]==0.30.0
pydantic==2.8.0
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4

# Database & Crypto
cryptography==43.0.0

# LLM & RAG
ollama==0.3.0
chromadb==0.5.0
sentence-transformers==3.1.0

```

```
# Forensics
pytsk3==20240805
yara-python==4.5.1
volatility3==2.7.0
python-registry==1.4.0
pefile==2023.2.7

# UI
PyQt6==6.7.0
pyautogui==0.9.54
openai-whisper==20231117   # local inference only
pyaudio==0.2.14
sounddevice==0.4.7

# Reporting
reportlab==4.2.2
python-docx==1.1.2

# Utils
psutil==6.0.0
pyudev==0.24.1
python-magic==0.4.27
coloredlogs==15.0.1
rich==13.8.0
```

13.5 Appendix E: Hackathon Demo Script (5 Minutes)

Time	Action	What Judges See
0:00	Open CyberHawk	Halide topo hero dashboard with "No Active Investigation"
0:30	Insert USB	Toast notification: "New USB Device: SanDisk 32GB" – BadUSB check instant
1:00	Select Full Auto	Gooey text morphs: "Initiating Full Automation Mode..."
1:15	Backup dialog	PyAutoGUI opens backup destination selector
1:45	Backup complete	SHA-256 verified, write-block applied, imaging starts
2:30	YARA match	Critical finding alert: "YARA: APT28_XAgent matched in loader.exe"
3:00	LLM Analysis	AI panel fills with citation-backed analysis, MITRE ATT&CK tag: T1059
3:30	Show tools grid	Ultra-quality showcase grid shows 25 tools, green LEDs
4:00	Evidence vault	Spatial product showcase 3D parallax showing case summary
4:30	Generate report	PDF report downloaded: 15 pages, chain of custody intact

Time	Action	What Judges See
5:00	Network check	Wireshark open – ZERO packets – full offline confirmed

Final Statement for Judges

CyberHawk is the **only forensic platform** that combines three things no existing tool does simultaneously: a 100% offline multi-model LLM reasoning engine, a court-admissible evidence chain with cryptographic integrity, and a consumer-grade UI beautiful enough to not require forensic training.
Built on proven open-source infrastructure (Servos, Orca, Orchestrata, Ollama) with production-quality UI components (21st.dev), and scaffolded by 713+ battle-tested Antigravity skills – this isn't a hackathon prototype. It's a **deployable forensics workstation**.

"Forensics for the Rest of Us."
Air-gappable. Intelligent. Automated.

CyberHawk Antigravity Build PRD v2.0

Team MoMoSapiens | CyberHack V4 2025 | Built with antigravity-awesome-skills v4.11.0

Skills: @security-auditing @rag-engineer @react-patterns @python-patterns @docker-expert @typescript-expert
UI: halide-topo-hero | gooey-text-morphing | ultra-quality-showcase-grid | spatial-product-showcase